

Basics of Using Cohort Prevalence

Kelly Li

Martin Lavalley

Contents

0.1	Overview	1
0.2	Connection Details	1
0.3	Defining cohorts	1
0.4	Step 1: Cohorts and periods of interest	3
0.5	Step 2: Prevalence analysis options	3
0.6	Step 3: Run analyses	4
0.7	Step 4: Export	5
0.8	Running Multi-Dimensional Prevalence Experiments	5

0.1 Overview

This tutorial walks you through running a `CohortPrevalence` for a single yearly prevalence analysis.

0.2 Connection Details

We use the `ClinicalCharacteristics` package to specify execution settings for the analysis with information on our databases and schemas of interest, and connect to the databases using `DatabaseConnector`.

```
# Create Connection details via DatabaseConnector
connectionDetails <- DatabaseConnector::createConnectionDetails(
  dbms = "dbms",
  user = "ulysses",
  password = "shh_secret"
)

# create execution Settings
executionSettings <- ClinicalCharacteristics::createExecutionSettings(
  connectionDetails = connectionDetails,
  cdmDatabaseSchema = cdmDatabaseSchema, # schema containing patient data
  workDatabaseSchema = workDatabaseSchema, # schema to write to
  tempEmulationSchema = tempEmulationSchema, # schema to write temporary tables to
  cohortTable = cohortTable, # table on the workDatabaseSchema containing cohort data
  cdmSourceName = cdmSourceName # human-readable database source name
)

connection <- DatabaseConnector::connect(connectionDetails)
```

0.3 Defining cohorts

In this test case, we use `CapR` to generate a cohort of hypertension patients in our OMOP database.

```
library(CapR)
```

```

# make Capr concept set for Esse
hypertensiveDisorder <- cs(
  descendants(316866),
  name = "Hypertensive disorder"
)
#fill out concept set details from vocabulary
hypertensiveDisorder <- getConceptSetDetails(
  x = hypertensiveDisorder,
  con = connection,
  vocabularyDatabaseSchema = executionSettings$cdmDatabaseSchema
)

# make Capr cohort all by all to calculate prevalence
cohort <- cohort(
  entry = entry(
    conditionOccurrence(conceptSet = hypertensiveDisorder),
    primaryCriteriaLimit = "All"
  ),
  attrition = attrition(
    expressionLimit = "All"
  ),
  exit = exit(
    endStrategy = fixedExit(index = "start", offsetDays = 0)
  )
)

# prepare cohort for CohortGenerator
json <- compile(cohort, pretty = TRUE)
sql <- CirceR::buildCohortQuery(
  expression = CirceR::cohortExpressionFromJson(json),
  options = CirceR::createGenerateOptions(generateStats = FALSE)
)

cohortDefinitionSet <- data.frame(
  cohortId = 316866,
  cohortName = "hypertension",
  json = json,
  sql = sql
)

# build cohort tables for generation
cohortTableNames <- CohortGenerator::getCohortTableNames(cohortTable = executionSettings$cohortTable)

CohortGenerator::createCohortTables(
  connectionDetails = connectionDetails,
  cohortDatabaseSchema = executionSettings$workDatabaseSchema,
  cohortTableNames = cohortTableNames
)

# generate cohorts
CohortGenerator::generateCohortSet(
  connectionDetails = connectionDetails,
  cohortDatabaseSchema = executionSettings$workDatabaseSchema,

```

```

tempEmulationSchema = executionSettings$tempEmulationSchema,
cohortTableNames = cohortTableNames,
cdmDatabaseSchema = executionSettings$cdmDatabaseSchema,
cohortDefinitionSet = cohortDefinitionSet
)

# Check counts
cohortCounts <- CohortGenerator::getCohortCounts(
  connectionDetails = connectionDetails,
  cohortDatabaseSchema = executionSettings$workDatabaseSchema,
  cohortTable = cohortTableNames$cohortTable
)

cohortCounts

```

0.4 Step 1: Cohorts and periods of interest

The first step of the analysis is to specify the cohort of prevalent interest and the periods of interest. In order to do so, we create the R6 classes that define these analyses settings. We can select between yearly prevalence analyses or span prevalence analyses using `createYearlyRange` and `createSpan`. Span prevalence analyses are more flexible; you can choose to input a range of starting and ending years, or specific dates (i.e. '2025-06-30' to '2026-01-01'). By default, analyses where you only specify the year start and end on the first day of the year (i.e. 2016-2017 translates to '2016-01-01' to '2017-01-01').

Sometimes, we may want to do an analysis on a subpopulation of the overall database (i.e. prevalence of hypertension in sitagliptin users). In this analysis, we care about the entire population, and so we leave the `populationCohort` option NULL.

```

prevalentCohort <- createTargetCohort(cohortId = 1,
                                     cohortName = "Hypertension")
# Yearly prevalence for periods 2016-2017, 2017-2018, etc.
periodOfInterest <- createYearlyRange(range = c(2016:2020))

# Span prevalence for periods of interest 2016-2019, 2018-2019, and 2020-2025.
periodOfInterestSpan <- createSpan(startDates = c(2016, 2018, 2020),
                                   endDates = c(2019, 2019, 2025))

populationCohort <- NULL

```

0.5 Step 2: Prevalence analysis options

Next, we define the options specific to the prevalence analysis. The numerator and denominator are chosen together through `createPrevalenceType()`. `CohortPrevalence` uses operational definitions of prevalence from Rassen et al. Please see the vignette `prevalence` for definitions of the numerator and denominator choices.

A `PrevalenceType` bundles four settings:

- `prevalenceType`: one of "point_prevalence" (pn1 + pd1), "period_prevalence_pd2", "period_prevalence_pd3", or "period_prevalence_pd4".
- `lookBackDays`: how far back to look for a qualifying event; `Inf` means complete historical lookback.
- `mode`: "formal" anchors on `cohort_start_date`, "rough" anchors on `cohort_end_date`.
- `leadInDays`: days of continuous observation a person must accrue *before* the period of interest starts in order to enter the denominator. The default 0L keeps every observation period that overlaps the period of interest.

Beyond the prevalence type, we set the demographic constraints of the population, the rate multiplier, the strata variables, and whether we want to use only the first observation period.

```
analysisId <- 123 # Any unique integer ID to define this analysis

# Select the prevalence type (numerator + denominator), lookback, mode, and lead-in
prevalenceType <- createPrevalenceType(
  prevalenceType = "period_prevalence_pd3",
  lookBackDays = 99999L,
  mode = "formal",
  leadInDays = 0L
)

# Constrain the population by age and gender
demographicConstraints <- createDemographicConstraints(
  ageMin = 0,
  ageMax = 150,
  genderIds = c(8507, 8532)
)

# Set strata options - NULL means no stratification
strata <- NULL

# Set other specifications
useOnlyFirstObservationPeriod <- FALSE
multiplier <- 100000L

prevalenceAnalysisClass <- createCohortPrevalenceAnalysis(
  analysisId = analysisId,
  prevalentCohort = prevalentCohort,
  periodOfInterest = periodOfInterest,
  prevalenceType = prevalenceType,
  useOnlyFirstObservationPeriod = useOnlyFirstObservationPeriod,
  multiplier = multiplier,
  strata = strata,
  demographicConstraints = demographicConstraints,
  populationCohort = populationCohort,
  outputTypes = "prevalence"
)

# Review the assembled settings
prevalenceAnalysisClass$viewAnalysisInfo()
```

0.6 Step 3: Run analyses

The `prevalenceAnalysisClass` object is now a wrapped-up package of all our analysis specifications. Using this object, we can now run the analysis with `generatePrevalence` and our previously defined database connection settings. `generatePrevalence()` accepts either a single analysis object or a list of them, and returns a `PrevalenceResults` object.

```
# Results
results <- generatePrevalence(
  prevalenceAnalysisList = prevalenceAnalysisClass,
  executionSettings = executionSettings
)
```

```
# Inspect what came back
results
results$prevalence
```

0.7 Step 4: Export

Now, we can write the results into a portable bundle for sharing with `results$export()`. The bundle contains one .csv per result table plus a manifest, and can be read back with `loadPrevalenceResults()`. Sometimes, we want to externally review the SQL queries used to generate the prevalence objects. We can do this with `exportPrevalenceQuery`.

```
outputFolder <- here::here("results") |>
  fs::dir_create()

# Export results to a bundle of CSV files
results$export(outputFolder = outputFolder)

# Save SQL query
exportPrevalenceQuery(
  prevalenceAnalysisClass = prevalenceAnalysisClass,
  executionSettings = executionSettings,
  outputFolder = outputFolder
)
```

0.8 Running Multi-Dimensional Prevalence Experiments

For more complex analyses involving multiple cohorts, prevalence types, demographic constraints, and periods of interest, you can use the `CohortPrevalenceExperiment` class. This approach automatically generates all combinations of analysis dimensions through a Cartesian product expansion.

0.8.1 Define Analysis Dimensions

```
# Create experiment specification
exp <- CohortPrevalenceExperiment$new(
  name = "Hypertension Multi-Analysis Study",
  description = "Multiple prevalence types and demographic stratifications"
)

# Add cohort definitions
exp$addCohorts(tibble::tibble(
  cohortId = c(316866),
  cohortName = c("Hypertension")
))

# Add prevalence types (formal, rough, point, and period)
exp$addPrevalenceTypes(list(
  createPrevalenceType("point_prevalence", lookBackDays = 0L, mode = "formal"),
  createPrevalenceType("period_prevalence_pd3", lookBackDays = 365L, mode = "formal"),
  createPrevalenceType("period_prevalence_pd3", lookBackDays = 365L, mode = "rough"),
  # same definition as above, but requiring a year of prior observation
  createPrevalenceType("period_prevalence_pd3", lookBackDays = 365L, mode = "formal", leadInDays = 365L)
))
```

```

# Add demographic constraints (age ranges)
exp$addDemographicConstraints(list(
  createDemographicConstraints(ageMin = 18, ageMax = 150),
  createDemographicConstraints(ageMin = 65, ageMax = 150)
))

# Add periods of interest
exp$addPeriodsOfInterest(list(
  createYearlyRange(2016:2020)
))

# Settings shared by every analysis in the grid
exp$setCommonParameters(
  strata = c("age", "gender"),
  outputTypes = "prevalence",
  useOnlyFirstObservationPeriod = FALSE
)

# Review the grid before running anything
exp$viewDesign()

```

0.8.2 Execute All Analyses

```

# Materialize all analysis specifications
analyses <- exp$define()

# Execute all analyses
results <- generatePrevalence(
  prevalenceAnalysisList = analyses,
  executionSettings = executionSettings,
  captureSql = TRUE
)

#standardize results
results$standardizePrevalence(
  referencePopulation = usa_census_2020,
  ageRightTruncation = 70
)

# Export all results
results$export(outputFolder = outputFolder)

```